

Structural Design Patterns

Exam Templates, Mixed Patterns & Practice Problems

CSE 214 Exam Preparation

Contents

1	How to Use This Booklet	3
2	Pattern Identification Cheat Sheet	3
2.1	The Four Mental Pictures	4
3	Template 1 — Decorator	5
3.1	When to Use It	5
3.2	Core Structure	5
3.3	Exam Template	5
3.4	The Lines to Memorize	7
4	Template 2 — Composite	7
4.1	When to Use It	7
4.2	Core Structure	7
4.3	Exam Template	8
4.4	The Most Important Line	9
4.5	Typical Client	9
5	Template 3 — Adapter	10
5.1	When to Use It	10
5.2	Core Structure	10
5.3	Exam Template	10
6	Template 4 — Bridge	11
6.1	When to Use It	11
6.2	Core Structure	12
6.3	Generic Exam Template	12
6.4	The Line to Memorize	13
7	The Four Tiny Skeletons to Memorize	14
8	Mixed Pattern Template 1 — Bridge + Decorator	15
8.1	Mental Model	15
8.2	Reusable Skeleton	15
9	Mixed Pattern Template 2 — Adapter + Decorator	18
9.1	Mental Model	18
9.2	Reusable Skeleton	18
10	Mixed Pattern Template 3 — Composite + Decorator	20
11	Mixed Pattern Template 4 — Bridge + Adapter	22
12	Mixed Pattern Template 5 — Bridge + Composite	23

13 How to Analyze Mixed Patterns	24
14 Analysis of the Uploaded Lab Questions	24
14.1 A1 — Gift Shop	24
14.2 B1 — Gift Boxes	25
14.3 B2 — Grocery Packages	25
14.4 C1 — Legacy API	25
15 Eight-to-Ten Practice Problems	25
15.1 Practice 1 — Food Delivery Customization	26
15.2 Practice 2 — Nested Travel Packages	26
15.3 Practice 3 — Legacy Payment System	26
15.4 Practice 4 — Cloud Storage	27
15.5 Practice 5 — Shopping Cart + Optional Features	27
15.6 Practice 6 — Restaurant Packages	27
15.7 Practice 7 — Legacy Media API + Optional Processing	28
15.8 Practice 8 — Delivery System 2.0	28
15.9 Practice 9 — File Explorer	29
15.10 Practice 10 — Smart Notification System	29
16 Suggested Practice Strategy	29
17 30-Minute Exam Workflow	31
18 Highest-Priority Mixed Combinations	31
19 Final One-Page Mental Cheat Sheet	32

1. How to Use This Booklet

The goal of this booklet is not to memorize hundreds of lines of Java. Instead, memorize the **structural skeleton** of each pattern.

The four structural patterns covered in the course material are:

1. Decorator
2. Composite
3. Adapter
4. Bridge

The uploaded lab-test questions show that the coding questions are not simply “implement pattern X.” They are story-based problems where the first task is to recognize one or more patterns and then construct the required classes.

The four lab questions particularly suggest the following combinations:

- **A1: Bridge + Decorator**
- **B1: Composite + Decorator**
- **B2: Composite**
- **C1: Adapter + Decorator**

Exam Tip

In the exam, do not immediately start writing concrete classes. First translate the story into a structural diagram. Once the pattern is identified, write the memorized skeleton and then fill in the problem-specific names and operations.

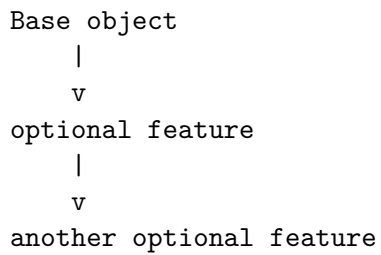
2. Pattern Identification Cheat Sheet

When reading a question, look for these clues.

Story clue	Likely pattern
“Add optional features”, “extra feature”, “can combine several features”, “without changing the original object”	Decorator
“Packages contain items/-packages”, “tree”, “nested”, “treat individual and group uniformly”	Composite
“Old/legacy class”, “cannot modify”, “incompatible interface”, “old system”	Adapter
“Two independent dimensions”, “both may grow independently”, “avoid combinations/classes explosion”	Bridge

2.1 The Four Mental Pictures

Decorator



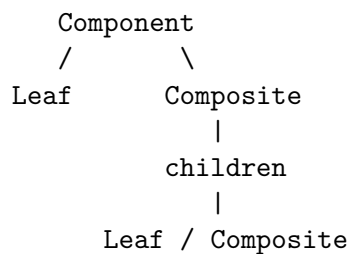
Example:

```

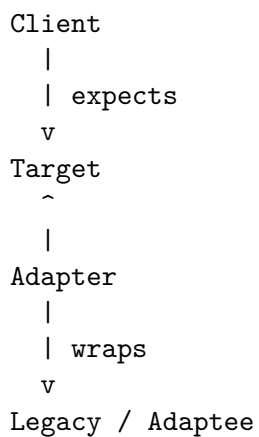
Coffee
+ Milk
+ Mocha
+ Whipped Cream

```

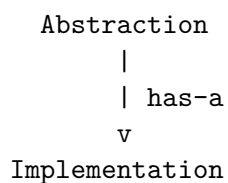
Composite



Adapter



Bridge



The important point is that the abstraction hierarchy and implementation hierarchy can grow independently.

3. Template 1 — Decorator

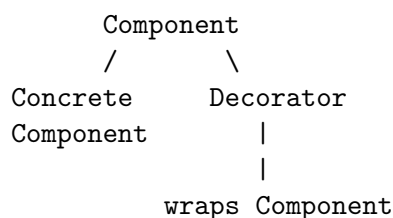
3.1 When to Use It

Use Decorator when a base object can receive optional behavior or features dynamically, and several features may be combined.

Typical words:

- optional
- add-on
- extra
- customization
- can be combined
- without modifying the original class

3.2 Core Structure



The decorator implements the same interface as the object it wraps.

3.3 Exam Template

```

1 // =====
2 // DECORATOR TEMPLATE
3 // =====
4
5 // Component
6 interface Component {
7     double getPrice();
8     String getDescription();
9 }
10
11
12 // Concrete Components
13 class ConcreteA implements Component {
14
15     private double price;
16
17     public ConcreteA(double price) {
18         this.price = price;
19     }
20
21     @Override
22     public double getPrice() {
23         return price;
24     }
25
26     @Override
27     public String getDescription() {

```

```
28     return "Concrete A";
29 }
30 }
31
32
33 class ConcreteB implements Component {
34
35     private double price;
36
37     public ConcreteB(double price) {
38         this.price = price;
39     }
40
41     @Override
42     public double getPrice() {
43         return price;
44     }
45
46     @Override
47     public String getDescription() {
48         return "Concrete B";
49     }
50 }
51
52
53 // Base Decorator
54 abstract class Decorator implements Component {
55
56     protected Component wrapped;
57
58     public Decorator(Component wrapped) {
59         this.wrapped = wrapped;
60     }
61 }
62
63
64 // Concrete Decorators
65 class DecoratorA extends Decorator {
66
67     public DecoratorA(Component wrapped) {
68         super(wrapped);
69     }
70
71     @Override
72     public double getPrice() {
73         return wrapped.getPrice() + /* extra cost */;
74     }
75
76     @Override
77     public String getDescription() {
78         return wrapped.getDescription() + " + Decorator A";
79     }
80 }
81
82
83 class DecoratorB extends Decorator {
84
85     public DecoratorB(Component wrapped) {
86         super(wrapped);
87     }
88 }
```

```

89  @Override
90  public double getPrice() {
91      return wrapped.getPrice() + /* extra cost */;
92  }
93
94  @Override
95  public String getDescription() {
96      return wrapped.getDescription() + " + Decorator B";
97  }
98  }

```

3.4 The Lines to Memorize

```

1  protected Component wrapped;
2
3  public Decorator(Component wrapped) {
4      this.wrapped = wrapped;
5  }

```

The client can repeatedly wrap the object:

```

1  Component x = new ConcreteA(100);
2
3  x = new DecoratorA(x);
4  x = new DecoratorB(x);
5
6  System.out.println(x.getPrice());

```

The object remains a `Component` throughout the chain.

4. Template 2 — Composite

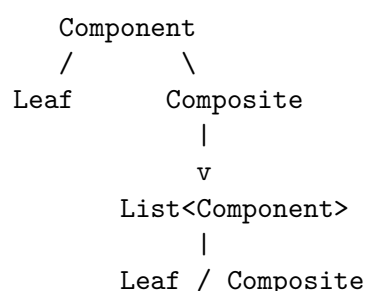
4.1 When to Use It

Use Composite when the question describes a hierarchy or tree in which an individual object and a group of objects should be treated uniformly.

Typical clues:

- package contains items
- package contains packages
- folder contains folders
- nested
- tree
- individual and group should behave uniformly

4.2 Core Structure



4.3 Exam Template

```
1 // =====
2 // COMPOSITE TEMPLATE
3 // =====
4
5 import java.util.ArrayList;
6 import java.util.List;
7
8
9 // Component
10 interface Item {
11
12     double getPrice();
13
14     void print(String indent);
15 }
16
17
18 // Leaf
19 class Product implements Item {
20
21     private String name;
22     private double price;
23
24     public Product(String name, double price) {
25         this.name = name;
26         this.price = price;
27     }
28
29     @Override
30     public double getPrice() {
31         return price;
32     }
33
34     @Override
35     public void print(String indent) {
36         System.out.println(
37             indent + name + " ($" + price + ")"
38         );
39     }
40 }
41
42
43 // Composite
44 class Package implements Item {
45
46     private String name;
47     private List<Item> items = new ArrayList<>();
48
49     public Package(String name) {
50         this.name = name;
51     }
52
53     public void add(Item item) {
54         items.add(item);
55     }
56
57     @Override
58     public double getPrice() {
59
```

```

60     double total = 0;
61
62     for (Item item : items) {
63         total += item.getPrice();
64     }
65
66     return total;
67 }
68
69 @Override
70 public void print(String indent) {
71
72     System.out.println(indent + name);
73
74     for (Item item : items) {
75         item.print(indent + " ");
76     }
77 }
78 }

```

4.4 The Most Important Line

```

1 private List<Item> items;

```

Because the list stores the **same interface** as the leaf:

```

Package
|
+-- Product
+-- Product
+-- Package
    |
    +-- Product
    +-- Product

```

A Composite can therefore contain another Composite.

4.5 Typical Client

```

1 Product burger = new Product("Burger", 8);
2 Product fries = new Product("Fries", 3);
3
4 Package lunch = new Package("Lunch Combo");
5
6 lunch.add(burger);
7 lunch.add(fries);
8
9 Package order = new Package("Order");
10
11 order.add(lunch);
12 order.add(new Product("Pizza", 10));
13
14 order.print("");
15 System.out.println(order.getPrice());

```

5. Template 3 — Adapter

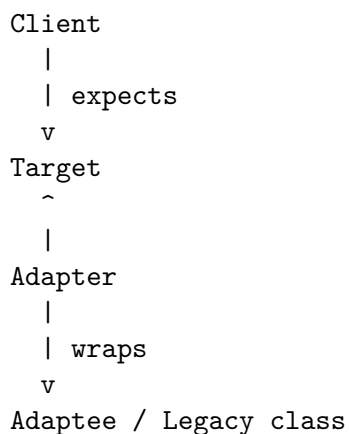
5.1 When to Use It

Use Adapter when existing code has the functionality you need but its interface does not match what the client expects.

The strongest clues are:

- legacy system
- cannot modify the old class
- old API
- incompatible interface
- convert one interface into another

5.2 Core Structure



5.3 Exam Template

```

1 // =====
2 // ADAPTER TEMPLATE
3 // =====
4
5 // Target
6 interface Target {
7
8     String getData();
9 }
10
11
12 // Adaptee
13 class LegacyService {
14
15     public String getOldData() {
16         return "legacy data";
17     }
18 }
19
20
21 // Adapter
22 class Adapter implements Target {
23
24     private LegacyService legacy;
  
```

```

25
26 public Adapter(LegacyService legacy) {
27     this.legacy = legacy;
28 }
29
30 @Override
31 public String getData() {
32
33     String oldData = legacy.getOldData();
34
35     // Convert old format to required format
36     String newData = /* conversion */;
37
38     return newData;
39 }
40 }

```

Client:

```

1 Target service =
2     new Adapter(new LegacyService());
3
4 System.out.println(service.getData());

```

Exam Tip

The Adapter does not modify the legacy class. It translates the legacy interface or output into the interface expected by the client.

6. Template 4 — Bridge

6.1 When to Use It

Bridge is the pattern to look for when there are **two independent dimensions of variation**.

For example:

Delivery Region:

- Local
- National
- International

Delivery Mode:

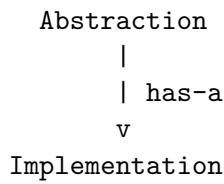
- Standard
- Express
- Priority

Without Bridge, the programmer may start creating:

- LocalStandard
- LocalExpress
- LocalPriority
- NationalStandard
- NationalExpress
- NationalPriority
- InternationalStandard
- InternationalExpress
- InternationalPriority

That becomes a class explosion.
 Bridge separates the two dimensions.

6.2 Core Structure



There are two separate class hierarchies.

6.3 Generic Exam Template

```

1 // =====
2 // BRIDGE TEMPLATE
3 // =====
4
5 // Implementation
6 interface Implementation {
7
8     double operationA();
9     String operationB();
10 }
11
12 // Concrete Implementations
13 class ImplementationA implements Implementation {
14
15     @Override
16     public double operationA() {
17         return 100;
18     }
19
20     @Override
21     public String operationB() {
22         return "Implementation A";
23     }
24 }
25
26
27 class ImplementationB implements Implementation {
28
29     @Override
30     public double operationA() {
31         return 200;
32     }
33
34     @Override
35     public String operationB() {
36         return "Implementation B";
37     }
38 }
39
40
41 // Abstraction
42 abstract class Abstraction {
43
44     protected Implementation implementation;
  
```

```
46
47 public Abstraction(Implementation implementation) {
48     this.implementation = implementation;
49 }
50
51 public abstract double operationA();
52
53 public abstract String operationB();
54 }
55
56
57 // Refined Abstractions
58 class RefinedA extends Abstraction {
59
60     public RefinedA(Implementation implementation) {
61         super(implementation);
62     }
63
64     @Override
65     public double operationA() {
66         return implementation.operationA();
67     }
68
69     @Override
70     public String operationB() {
71         return implementation.operationB();
72     }
73 }
74
75
76 class RefinedB extends Abstraction {
77
78     public RefinedB(Implementation implementation) {
79         super(implementation);
80     }
81
82     @Override
83     public double operationA() {
84         return implementation.operationA();
85     }
86
87     @Override
88     public String operationB() {
89         return implementation.operationB();
90     }
91 }
```

6.4 The Line to Memorize

```
1 protected Implementation implementation;
2
3 public Abstraction(Implementation implementation) {
4     this.implementation = implementation;
5 }
```

The abstraction delegates to its implementation.

7. The Four Tiny Skeletons to Memorize

If time is limited, memorize these instead of memorizing the complete templates.

Decorator

```
1 interface X { }
2
3 abstract class Decorator implements X {
4     protected X x;
5
6     Decorator(X x) {
7         this.x = x;
8     }
9 }
```

Composite

```
1 interface X { }
2
3 class Leaf implements X { }
4
5 class Composite implements X {
6     List<X> children = new ArrayList<>();
7
8     void add(X x) {
9         children.add(x);
10    }
11 }
```

Adapter

```
1 interface Target { }
2
3 class Adaptee { }
4
5 class Adapter implements Target {
6
7     Adaptee oldObject;
8
9     Adapter(Adaptee oldObject) {
10         this.oldObject = oldObject;
11     }
12 }
```

Bridge

```
1 interface Implementation { }
2
3 abstract class Abstraction {
4
5     protected Implementation implementation;
6
7     Abstraction(Implementation implementation) {
8         this.implementation = implementation;
9     }
10 }
```

Important

Do not memorize the names “Component”, “Decorator”, “Implementation”, etc. Memorize the relationships: Decorator wraps the same interface, Composite stores the same interface in a collection, Adapter wraps an incompatible class, and Bridge contains an implementation interface.

8. Mixed Pattern Template 1 — Bridge + Decorator

This is the highest-priority mixed combination because the uploaded A1 lab question is exactly this kind of problem.

A gift can receive optional wrapping, while delivery has two independently varying dimensions such as region and mode.

8.1 Mental Model



8.2 Reusable Skeleton

```

1 // =====
2 // BRIDGE + DECORATOR
3 // =====
4
5 // ----- Decorator side -----
6
7 interface Product {
8
9     double getPrice();
10
11     String getDescription();
12 }
13
14
15 class BasicProduct implements Product {
16
17     private double price;
18
19     public BasicProduct(double price) {
20         this.price = price;
21     }
22
23     @Override
24     public double getPrice() {
25         return price;
26     }

```

```
27
28     @Override
29     public String getDescription() {
30         return "Basic Product";
31     }
32 }
33
34
35 abstract class ProductDecorator implements Product {
36
37     protected Product product;
38
39     public ProductDecorator(Product product) {
40         this.product = product;
41     }
42 }
43
44
45 class ExtraFeature extends ProductDecorator {
46
47     public ExtraFeature(Product product) {
48         super(product);
49     }
50
51     @Override
52     public double getPrice() {
53         return product.getPrice() + 10;
54     }
55
56     @Override
57     public String getDescription() {
58         return product.getDescription() + " + Extra";
59     }
60 }
61
62
63 // ----- Bridge side -----
64
65 interface Platform {
66
67     double calculate(int input);
68
69     String getTime();
70 }
71
72
73 class PlatformA implements Platform {
74
75     @Override
76     public double calculate(int input) {
77         return input;
78     }
79
80     @Override
81     public String getTime() {
82         return "Normal";
83     }
84 }
85
86
87 class PlatformB implements Platform {
```

```

88
89  @Override
90  public double calculate(int input) {
91      return input + 20;
92  }
93
94  @Override
95  public String getTime() {
96      return "Slow";
97  }
98 }
99
100
101 abstract class Feature {
102
103     protected Platform platform;
104
105     public Feature(Platform platform) {
106         this.platform = platform;
107     }
108
109     public abstract double calculate(int input);
110
111     public abstract String getTime();
112 }
113
114
115 class FeatureA extends Feature {
116
117     public FeatureA(Platform platform) {
118         super(platform);
119     }
120
121     @Override
122     public double calculate(int input) {
123         return platform.calculate(input) + 10;
124     }
125
126     @Override
127     public String getTime() {
128         return platform.getTime();
129     }
130 }

```

Client:

```

1 Product p = new BasicProduct(100);
2
3 p = new ExtraFeature(p);
4
5 Feature x =
6     new FeatureA(new PlatformB());
7
8 double total =
9     p.getPrice() + x.calculate(50);

```

Exam Tip

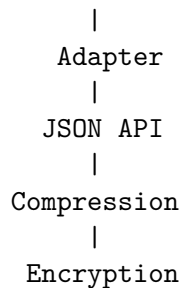
The two patterns are independent here. Decorator handles optional product features; Bridge handles the two delivery dimensions. Do not merge them into one giant inheritance tree.

9. Mixed Pattern Template 2 — Adapter + Decorator

This combination is especially important because the uploaded C1 question requires an unchanged legacy XML service, an adapter to the new API, and optional transformations such as compression and encryption.

9.1 Mental Model

Legacy XML Service



9.2 Reusable Skeleton

```

1 // ----- Target -----
2 interface Response {
3
4     String getBody();
5 }
6
7
8 // ----- Legacy -----
9 class LegacyService {
10
11     public String getXML() {
12         return "<user>...</user>";
13     }
14 }
15
16
17 // ----- Adapter -----
18 class XMLToJSONAdapter implements Response {
19
20     private LegacyService service;
21
22     public XMLToJSONAdapter(LegacyService service) {
23         this.service = service;
24     }
25
26     @Override
27     public String getBody() {
28
29         String xml = service.getXML();
30
31         return convertXMLtoJSON(xml);
32     }
33
34     private String convertXMLtoJSON(String xml) {
35         return "{ \"converted\": true }";
36     }
37 }
38
39

```

```

40 // ----- Decorator -----
41 abstract class ResponseDecorator implements Response {
42
43     protected Response response;
44
45     public ResponseDecorator(Response response) {
46         this.response = response;
47     }
48 }
49
50
51 // ----- Encryption -----
52 class EncryptionDecorator extends ResponseDecorator {
53
54     public EncryptionDecorator(Response response) {
55         super(response);
56     }
57
58     @Override
59     public String getBody() {
60
61         String data = response.getBody();
62
63         return encrypt(data);
64     }
65
66     private String encrypt(String data) {
67         return "ENCRYPTED(" + data + ")";
68     }
69 }
70
71
72 // ----- Compression -----
73 class CompressionDecorator extends ResponseDecorator {
74
75     public CompressionDecorator(Response response) {
76         super(response);
77     }
78
79     @Override
80     public String getBody() {
81
82         String data = response.getBody();
83
84         return compress(data);
85     }
86
87     private String compress(String data) {
88         return "COMPRESSED(" + data + ")";
89     }
90 }

```

Client:

```

1 Response response =
2     new XMLToJSONAdapter(
3         new LegacyService()
4     );
5
6 response =
7     new CompressionDecorator(response);
8

```

```

9 response =
10     new EncryptionDecorator(response);

```

The nested form is:

```

1 Response response =
2     new EncryptionDecorator(
3         new CompressionDecorator(
4             new XMLToJSONAdapter(
5                 new LegacyService()
6             )
7         )
8     );

```

This works because each decorator implements the same `Response` interface.

10. Mixed Pattern Template 3 — Composite + Decorator

This is a strong candidate for problems similar to the uploaded B1 question.

Composite handles the package hierarchy. Decorator handles optional packaging or other enhancements.

```

1 // ----- Composite -----
2 interface PackageItem {
3
4     double getPrice();
5
6     void print(String indent);
7 }
8
9
10 class Gift implements PackageItem {
11
12     private String name;
13     private double price;
14
15     public Gift(String name, double price) {
16         this.name = name;
17         this.price = price;
18     }
19
20     @Override
21     public double getPrice() {
22         return price;
23     }
24
25     @Override
26     public void print(String indent) {
27         System.out.println(indent + name);
28     }
29 }
30
31
32 class GiftPackage implements PackageItem {
33
34     private String name;
35     private List<PackageItem> items = new ArrayList<>();
36
37     public GiftPackage(String name) {
38         this.name = name;
39     }

```

```

40
41     public void add(PackageItem item) {
42         items.add(item);
43     }
44
45     @Override
46     public double getPrice() {
47
48         double total = 0;
49
50         for (PackageItem item : items) {
51             total += item.getPrice();
52         }
53
54         return total;
55     }
56
57     @Override
58     public void print(String indent) {
59
60         System.out.println(indent + name);
61
62         for (PackageItem item : items) {
63             item.print(indent + " ");
64         }
65     }
66 }
67
68
69 // ----- Decorator -----
70 abstract class PackagingDecorator implements PackageItem {
71
72     protected PackageItem item;
73
74     public PackagingDecorator(PackageItem item) {
75         this.item = item;
76     }
77 }
78
79
80 class PremiumBox extends PackagingDecorator {
81
82     public PremiumBox(PackageItem item) {
83         super(item);
84     }
85
86     @Override
87     public double getPrice() {
88         return item.getPrice() + 15;
89     }
90
91     @Override
92     public void print(String indent) {
93         item.print(indent);
94         System.out.println(indent + "Premium Gift Box");
95     }
96 }

```

Client example:

```

1 GiftPackage p = new GiftPackage("My Package");
2

```

```

3 p.add(new Gift("Chocolate", 10));
4 p.add(new Gift("Mug", 8));
5
6 PackageItem finalPackage =
7     new PremiumBox(p);

```

11. Mixed Pattern Template 4 — Bridge + Adapter

This combination is less obvious but useful for harder questions.

Suppose a system has two independently varying dimensions, while one of the underlying implementations comes from an incompatible legacy API.

Application Feature

|

Bridge

|

Implementation Interface

|

Adapter

|

Legacy / Third-party API

Example skeleton:

```

1 interface Platform {
2     void pay();
3 }
4
5
6 // Legacy / third-party
7 class OldPaymentAPI {
8     public void makePaymentInOldWay() { }
9 }
10
11
12 // Adapter
13 class PaymentAdapter implements Platform {
14
15     private OldPaymentAPI oldAPI;
16
17     public PaymentAdapter(OldPaymentAPI oldAPI) {
18         this.oldAPI = oldAPI;
19     }
20
21     @Override
22     public void pay() {
23         oldAPI.makePaymentInOldWay();
24     }
25 }
26
27
28 // Bridge abstraction
29 abstract class PaymentMethod {
30
31     protected Platform platform;
32
33     public PaymentMethod(Platform platform) {
34         this.platform = platform;
35     }

```

```

36     abstract void pay();
37 }
38

```

Then a refined abstraction can use:

```

1 new CreditCard(
2     new PaymentAdapter(
3         new OldPaymentAPI()
4     )
5 );

```

Do not memorize this combination as a fixed recipe. Recognize the two relationships and combine the basic skeletons.

12. Mixed Pattern Template 5 — Bridge + Composite

This is a harder variation.

Example idea:

```

FileSystemItem
|
Composite
/      \
File    Folder
        |
        List<Item>

```

AND

```

FileSystemItem
|
Bridge
|
StoragePlatform
/      |      \
Local  Cloud  Network

```

The core Bridge portion can begin with:

```

1 interface StoragePlatform {
2     void save(String data);
3 }
4
5
6 abstract class FileSystemItem {
7
8     protected StoragePlatform storage;
9
10    public FileSystemItem(StoragePlatform storage) {
11        this.storage = storage;
12    }
13 }

```

Then the Composite portion gives the folder a collection of `FileSystemItem` objects.

The important exam skill is recognizing that one pattern does not prevent another pattern from existing in a different part of the design.

13. How to Analyze Mixed Patterns

When a question looks complicated, split the story into independent requirements.

For example:

“Users can add encryption and compression in any combination.”

Immediately write:

Decorator

If the same question also says:

“The existing legacy service cannot be modified and returns XML.”

write:

Adapter

Therefore:

Adapter + Decorator

Similarly:

- “Package contains package” → Composite
- “Optional package features” → Decorator
- “Two independent axes” → Bridge
- “Legacy incompatible API” → Adapter

14. Analysis of the Uploaded Lab Questions

The following mapping is based on the uploaded lab-test questions.

14.1 A1 — Gift Shop

The question has two major requirements.

First, gifts can optionally be wrapped, which is naturally a Decorator requirement.

Second, delivery regions and delivery modes must be extendable independently. That is the key Bridge requirement.

Therefore:

Bridge + Decorator

The important architecture is:

```

Gift
|
Decorator
|
GiftWrap

```

```

Delivery
|
Bridge
/   \
Region Mode

```

The question explicitly states that regions and modes should be independently extendable, which is the strongest Bridge clue.

14.2 B1 — Gift Boxes

The question describes individual gifts and user-created boxes. A box can contain individual gifts and other boxes.

Therefore:

Composite

The optional premium or eco-friendly packaging features naturally suggest Decorator as the second structural concern.

Therefore, a strong design is:

Composite + Decorator

14.3 B2 — Grocery Packages

The question explicitly requires grocery packages to contain items or other packages, while orders should treat items, set menus, and packages uniformly.

Therefore:

Composite

The key structure is:

```

Item
 / \
Leaf Composite
    |
    +--- Item
    +--- Item
    +--- Composite
  
```

14.4 C1 — Legacy API

The question says the old XML service must remain unchanged while a new client expects a JSON-style API. This is Adapter.

Then encryption and compression can be applied in arbitrary combinations. That is Decorator.

Therefore:

Adapter + Decorator

The architecture is:

```

Legacy XML Service
  |
  Adapter
  |
  JSON
  |
  Decorator chain
 /      \
Compression  Encryption
  
```

15. Eight-to-Ten Practice Problems

These are designed to resemble the style and complexity of the uploaded lab questions. For each one, first identify the pattern(s), then draw the class structure, then code it.

15.1 Practice 1 — Food Delivery Customization

Difficulty:

A restaurant sells burgers, pizzas and sandwiches. Customers can optionally add cheese, extra sauce, or bacon to any food item. Each addition changes both price and description.

Additions can be combined in any order without modifying the original food classes.

Example:

```
Burger = $8
+ Cheese = $2
+ Bacon = $3
```

```
Final = $13
```

Pattern: _____

15.2 Practice 2 — Nested Travel Packages

Difficulty: 1/2

A travel company sells individual activities such as hiking, museum visits and boat rides. It also sells packages containing multiple activities.

A package may contain:

- individual activities
- other packages

The system must display the complete hierarchy and calculate total price.

Example:

```
Europe Package
  Museum
  City Package
    Boat Ride
    Hiking
```

Pattern: _____

15.3 Practice 3 — Legacy Payment System

Difficulty: 1/2

A company has:

```
1 class LegacyPaymentSystem {
2     void makeTransaction(String account, double amount) {
3         // existing implementation
4     }
5 }
```

The new application expects:

```
1 interface Payment {
2     void pay(double amount);
3 }
```

The legacy class cannot be modified.

Implement the system so the new application can use the old payment system.

Pattern: _____

15.4 Practice 4 — Cloud Storage

Difficulty:

A file application supports the following storage platforms:

- Local Disk
- Cloud Storage
- Network Storage

At the same time, users can interact with:

- Basic File
- Secure File
- Compressed File

The company expects to add more storage platforms and more file types independently.

Avoid creating classes such as:

```
SecureCloudFile
SecureNetworkFile
CompressedCloudFile
...
```

Implement the architecture.

Pattern: _____

15.5 Practice 5 — Shopping Cart + Optional Features

Difficulty:

An online store sells laptops, phones and tablets. Customers can optionally select:

- Extended Warranty: +\$50
- Gift Packaging: +\$10
- Express Handling: +\$20

Any number of these features may be selected.

The system must calculate final price and display the complete description.

No product class should be modified when a new optional feature is introduced.

Pattern: _____

15.6 Practice 6 — Restaurant Packages

Difficulty:

A restaurant sells individual food items and packages. A package may contain individual items and other packages.

Customers may also choose:

- Premium Packaging: +\$12
- Eco Packaging: +\$7

The final system must:

1. display the complete nested package hierarchy,
2. calculate the final price,
3. allow different packaging options without modifying package classes.

Patterns: _____ + _____

15.7 Practice 7 — Legacy Media API + Optional Processing

Difficulty: 1/2

An old media service returns XML and cannot be modified.

The new application expects:

```
1 interface MediaResponse {  
2     String getData();  
3 }
```

The XML must first be converted to JSON.

After conversion, users may request any combination of:

- Compression
- Encryption
- Watermark

The order of processing must be preserved.

The company expects to add more processing features later.

Do not create classes for every possible combination.

Patterns: _____ + _____

15.8 Practice 8 — Delivery System 2.0

Difficulty: 1/2

A courier company supports regions:

- City
- Country
- International

and delivery modes:

- Standard
- Express
- Priority

Each region has its own base pricing and default delivery time. Each mode changes pricing and time differently.

The company expects to introduce more regions and more modes independently.

Customers can also optionally request:

- Fragile Handling: +\$5
- Insurance: +\$15

These options can be combined.

Patterns: _____ + _____

15.9 Practice 9 — File Explorer

Difficulty:

A file explorer has File and Folder objects. A folder may contain files and other folders.

The system must:

- display the hierarchy,
- calculate total size,
- treat files and folders uniformly.

Additionally, the application supports:

- Local Storage
- Cloud Storage

The same file/folder system should work with either storage platform, and future storage platforms should be added independently.

Patterns: _____ + _____

15.10 Practice 10 — Smart Notification System

Difficulty:

A notification application supports:

- Email
- SMS
- Push Notification

The actual communication infrastructure may be:

- AWS
- Local Server
- Third-Party API

Both notification types and infrastructure providers may be expanded independently.

Additionally, a notification may optionally be:

- Encrypted
- Compressed
- Logged

The operations must be chainable.

Patterns: _____ + _____

16. Suggested Practice Strategy

For each practice question, use the following process.

Step 1 — Pattern Identification

Write only the pattern names first.

For example:

Adapter + Decorator

Step 2 — Draw the Skeleton

For Adapter:

```

Target
  ^
Adapter
  |
Adaptee

```

For Decorator:

```

Component
  ^
Decorator
  |
wrapped Component

```

For Composite:

```

Component
 /    \
Leaf   Composite
      |
      List<Component>

```

For Bridge:

```

Abstraction
  |
Implementation

```

Step 3 — Write Interfaces First

Do not begin with the main method.

Write:

1. interface(s),
2. abstract class(es),
3. concrete classes,
4. client/main.

Step 4 — Build the Required Cases

Only after the architecture works should you create the examples requested by the question.

Step 5 — Final Requirement Check

Before submitting, check:

- Does every required operation work?
- Can all requested combinations be constructed?
- Did I avoid modifying legacy code where prohibited?
- Can a Composite contain another Composite?
- Can a Decorator wrap another Decorator?
- Can Bridge dimensions vary independently?
- Does the main method demonstrate the requested cases?

17. 30-Minute Exam Workflow

If the coding question is around 30 minutes, a practical workflow is:

Time	Action
0–2 min	Identify pattern(s) from story clues.
2–5 min	Draw the structural skeleton.
5–20 min	Write interfaces, abstract classes, concrete classes.
20–25 min	Build the required client/main cases.
25–30 min	Test requirements, calculations, combinations, and compilation.

Exam Tip

Do not spend the first ten minutes writing concrete classes. The fastest approach is to recognize the structure first and immediately write the corresponding template.

18. Highest-Priority Mixed Combinations

Based specifically on the uploaded lab questions, prioritize your practice in this order:

Priority	Combination	Reason
1	Bridge + Decorator	Directly reflected in A1.
2	Adapter + Decorator	Directly reflected in C1.
3	Composite + Decorator	Strongly suggested by B1.
4	Composite	Directly reflected in B2.
5	Composite + Bridge	Useful harder variation.
6	Bridge + Adapter	Less obvious but possible.
7	Three-pattern combinations	Possible, but do not prioritize over the above.

Important

Do not try to memorize a separate template for every possible combination. Learn to compose the four basic skeletons. A mixed-pattern problem is usually just two or more independent structural requirements in the same story.

19. Final One-Page Mental Cheat Sheet

Decorator

Question says:

optional / add-on / extra / combine

Remember:

same interface + wraps same interface

```
interface X { }
```

```
abstract class D implements X {  
    protected X x;  
    D(X x) { this.x = x; }  
}
```

Composite

Question says:

contains / nested / tree / package inside package

Remember:

same interface + children list

```
interface X { }
```

```
class Composite implements X {  
    List<X> children;  
}
```

Adapter

Question says:

legacy / incompatible / cannot modify / convert interface

Remember:

Target + Adapter + Adaptee

```
interface Target { }
```

```
class Adapter implements Target {  
    Adaptee old;  
}
```

Bridge

Question says:

two independent dimensions / both grow independently

Remember:

two hierarchies + has-a

```
interface Implementation { }
```

```
abstract class Abstraction {
    protected Implementation implementation;
}
```

Mixed Patterns

```
optional features
    +
two independent dimensions
    =
Decorator + Bridge
```

```
legacy incompatible API
    +
optional transformations
    =
Adapter + Decorator
```

```
nested packages
    +
optional package features
    =
Composite + Decorator
```

```
nested hierarchy
    +
independent implementation dimension
    =
Composite + Bridge
```

Final Exam Reminder

The most important skill is not memorizing the exact variable names in these templates.

Memorize the relationships:

Pattern	Relationship to remember
Decorator	Same interface + wraps another object of that interface
Composite	Same interface + Composite stores children of that interface
Adapter	Target interface + wraps incompatible Adaptee
Bridge	Abstraction contains Implementation; two dimensions vary independently

Once the relationship is recognized, the rest of the code is mostly filling in the names, fields, calculations, and required behavior from the question.